National University of Lesotho

Department of Mathematics and Computer Science

CS4433 Software Engineering

MODERATED LEARNING MANAGEMENT SYSTEM

Final Report - Phases 1 to 8 (Release v1.1)

Student: Molapo Setsumi

Student no.: 202322042

Repository: github.com/molaposetsomi5/SC4433-MLMS

Submission date: 7 November 2026

1. Title Page, Abstract and Table of Contents

Abstract

This report describes the full development of the Moderated Learning Management System (MLMS), a menu-driven C++17 application for a secondary school. The system supports four roles (Administrator, Lecturer, Head of Department and Student) and enforces one key rule: no student sees a mark until the Head of Department has moderated and released it. It applies a 10% per day late penalty for up to three days, supports individual and class-wide extensions, keeps every version of a resubmission, and records every mark change in an append-only audit log.

The project followed the Waterfall model with V-Model verification. Planning, requirements and design were completed in Phases 1 to 3 and are summarised here, with the full phase documents linked from the repository. The system was implemented with a layered architecture, SQLite persistence behind repository interfaces, and Argon2id password hashing through libsodium. It is verified by 128 automated GoogleTest tests with 95.9% line coverage of the business-logic layer, a GitHub Actions pipeline, cppcheck and clang-tidy with zero remaining findings, and a performance test showing an 80 ms login with 800 students stored. Release v1.0 was followed by change request CR1 (class-wide extensions), delivered as release v1.1 with full regression testing.

Repository: https://github.com/molaposetsomi5/SC4433-MLMS

Releases: v1.0 | v1.1 | **CI runs:** GitHub Actions

Demonstration video: docs/video

Table of Contents

1. Title Page, Abstract and Table of Contents....1
2. Introduction....3
3. Project Management Plan....3
4. Requirements Engineering....4
5. Software Design Document....6
6. Implementation....8
7. Verification and Validation....9
8. Configuration Management and Quality Assurance....10
9. Deployment....11
10. Maintenance - Change Request CR1....11
11. Professional, Legal and Ethical Issues....12
12. Conclusion....13
13. References....13
14. Appendices....14

2. Introduction

A secondary school asked for a Learning Management System to manage courses, assignments and assessment. The case study names four types of user and a set of stakeholder statements that are incomplete and in places conflict with each other. The most important conflict is between the lecturer, who wants students to see marks as soon as work is marked, and the Principal and Head of Department (HoD), who want every mark checked before release. The school's IT officer adds that the computers are old, the internet is unreliable, and the system must protect passwords and stay fast with 800 students.

The aim of this project was to take that case study through the whole software engineering life cycle and deliver a working, tested system. The deliverables are this report, the phase documents (plan, requirements, design), the source code with its build guide, the test and quality evidence, deployment documents, and the change request analysis. Everything is stored in one Git repository so that the artefacts stay together and can be traced to each other.

- **Scope**

In scope are user and course administration, enrolment, assignments, submissions and resubmissions, extensions, marking and feedback, moderation, release, and the audit log. Out of scope, as stated in the brief, are payments, video conferencing, automated grading, plagiarism detection, and web or mobile interfaces. The system is a console application for a single school computer.

- **Development model**

The Waterfall model was used because the brief sets fixed phases with gate reviews, and the requirements could be agreed early. The V-Model was used for verification: each left-hand phase (requirements, architecture, detailed design) has a matching test level (acceptance and system tests, integration tests, unit tests). Section 7 shows this mapping.

3. Project Management Plan

The full plan is in the Phase 1 document. This section summarises the decisions it made and how the plan held up in practice.

Full document: Phase 1 Project Management Plan (PDF) ↗️

- **Feasibility study**

Area	Finding	Conclusion
Technical	Mature free tools (C++17, CMake, SQLite, libsodium, GoogleTest, cppcheck). SQLite needs no server and handles the data of 800 students easily.	Feasible, with RAII wrappers for SQLite resources
Operational	Mirrors the existing workflow: lecturers mark, HoDs moderate, students see released marks. Staff are not IT experts, so menus must be simple.	Feasible if the interface stays simple
Economic	No licence, hosting or hardware costs. Replaces paper or spreadsheet marking and gives a permanent audit trail.	Feasible with no cash outlay
Legal	Stores personal data of minors, so Lesotho's Data Protection Act 2011 applies. Tool licences (public domain, ISC, BSD, GPL for development only) permit school use.	Feasible if privacy and security are Must requirements

Area	Finding	Conclusion
Schedule	105 hours over 7 weeks; Weeks 2-4 run above the 15-hour average.	Feasible with strict scope control

The recommendation was to go ahead, on four conditions: keep the interface simple, treat security as a Must, hold the scope to the brief, and write the report as each phase ends.

- **Work breakdown structure and schedule**

The work was split into 9 phases and 39 work packages totalling 105 hours. The Gantt chart (Figure 1) shows the dependencies, the milestones (plan submitted, Gates 1-3, releases v1.0 and v1.1) and the critical path, which runs from the feasibility study through requirements, design, implementation, system testing and release to the final report. Because this is a Waterfall project, most tasks lie on the critical path, which is why the plan included a one-day buffer before each gate and a slippage rule: drop Could, then Should requirements before touching any Must requirement.

Figure 1. Gantt chart with dependencies, milestones and critical path. View diagram ↗️

- **Risk management**

The initial register had 14 risks across project, technical and people categories, each with likelihood, impact, exposure, mitigation and owner. The highest were stakeholder conflicts (R1) and schedule overrun (R2) at 16, then security flaws (R5) and a failed Gate 2 (R9) at 15. The register was re-scored at each gate. By Gate 3 the requirement, design and coverage risks were closed, and two new risks were added: failing to explain the code at the viva (R15) and the change request breaking existing features (R16). The slippage rule was used once: the Could requirement FR-20 (release notifications) was dropped.

Evidence: Updated risk register (Gates 0-3) ↗️

4. Requirements Engineering

The full stakeholder analysis, SRS and use case specifications are in the Phase 2 document. The SRS follows the structure of ISO/IEC/IEEE 29148.

Full document: Phase 2 Requirements Engineering (PDF) ↗️

- **Stakeholder analysis**

Six stakeholders were identified. Four are system roles (Administrator, Lecturer, HoD, Student). The Principal and the IT officer are external stakeholders: they do not log in, but their policies and constraints decide whether the system is accepted. The Principal and IT officer have high influence, so their statements about mark visibility, offline use and password protection became Must requirements.

- **Conflict and ambiguity resolution**

Fifteen conflicts and ambiguities (C1-C15) were found and resolved. The most important ones are shown below; each resolution became a requirement and later a test.

ID	Problem	Resolution
C1	Lecturer wants marks visible at once; Principal and HoD want them checked first.	Moderation comes first. The lecturer sees their own marks immediately; the HoD releases a whole assignment in one step so release is quick.
C2	Early marking would lock a student out of resubmitting.	A submission can only be marked after its effective deadline.
C3	"10% per day" - 10% of what, and what is a day?	A day is any started 24-hour period. Exactly at the deadline is on time. 10% of the maximum mark per day, floor of zero.
C4	"Zero after three days" vs "late work should not be rejected".	Late work is always accepted and recorded; after day 3 the final mark is 0 but feedback can still be given.
C5	Does an extension bypass the penalty?	An extension moves that student's effective deadline; the penalty clock starts from it. A reason is mandatory.
C6	How many resubmissions, and what happens to old versions?	Unlimited until the effective deadline while unmarked; all versions kept, the latest is marked.
C7	Can the HoD change marks or only approve?	Approve, amend (reason required) or return to the lecturer. Every change is audited.
C11	"Respond quickly" is not measurable.	Login and assignment list within 2 seconds with 800 students stored.
C13	No statement from the administrator.	Requirements derived from the case study; the administrator manages accounts but cannot see marks (least privilege).

- **Software Requirements Specification**

The SRS contains 20 functional requirements (FR-01 to FR-20) and 10 non-functional requirements (NFR-01 to NFR-10). Each has a unique ID, a MoSCoW priority and an acceptance criterion. The non-functional requirements are measurable; for example NFR-01 says login and opening the assignment list must take under 2 seconds with 800 students stored, and NFR-03 says only salted hashes from a recognised library may be stored. Table 1 lists the main ones; the complete list with traceability is in Appendix A.

ID	Requirement (short form)	Priority
FR-01, FR-02	Login with username and password; every operation restricted by role	Must
FR-03 - FR-05	Administrator manages users, departments, courses, lecturer allocation and enrolment	Must
FR-06, FR-07	Lecturer creates assignments; student sees assignments and effective deadlines	Must
FR-08, FR-09	Submit and resubmit before the deadline; late work accepted with 10%/day penalty, zero after 3 days	Must
FR-10	Lecturer grants an individual extension with a reason	Must
FR-11 - FR-13	Marking with feedback; HoD moderation (approve / amend / return); release of approved marks only	Must
FR-14, FR-18	Audit trail of every mark change (who, when, old, new, reason); scoped audit access	Must

ID	Requirement (short form)	Priority
FR-20	Notify students of newly released marks	Could (dropped)
NFR-01	Login and assignment list < 2 s with 800 students	Must
NFR-02, NFR-03	Works offline; passwords stored only as salted hashes	Must
NFR-05, NFR-06	PK/FK/CHECK constraints and transactions; append-only audit	Must

- **Use case model**

The use case diagram (Figure 2) covers all four actors, with a generalised User actor for Authenticate and Change password. It uses <<include>> for behaviour that always happens (recording the audit entry, applying the late penalty) and <<extend>> for optional behaviour (resubmitting before the deadline extends Submit assignment). Three use cases are fully dressed in the Phase 2 document with preconditions, main flow, alternative and exception flows, postconditions and business rules: UC-01 Submit assignment, UC-02 Mark submission and provide feedback, and UC-03 Moderate and release marks. These three specifications were later used to derive the system tests in Section 7.

Figure 2. Use case diagram (v1.1, includes CR1). View diagram ↗️

5. Software Design Document

The full design is in the Phase 3 document. The diagrams below were redrawn from the final code so that the design and the implementation match exactly; differences from the Phase 3 design are explained at the end of this section.

Full document: Phase 3 System Design (PDF) ↗️

- **Architecture**

A layered architecture was chosen with five layers: presentation (console menus), application services, domain rules, data access (repositories) and SQLite storage. Each layer only calls the layer below it. The console never runs SQL, and the domain rules do not depend on the console or the database, so they can be unit-tested on their own. A composition root (AppContext) creates every object once and passes dependencies in through constructors.

Rejected alternative: Model-View-Controller (MVC) was considered. MVC is useful when there are many views or a complex graphical interface. This system has one simple console view per role, so MVC would add controller classes without real benefit. The layered style gives the same separation of presentation, logic and storage with less structure, and it maps directly onto the folders in the source code.

Figure 3. Layered architecture and component diagram. View diagram ↗️

- **Class diagram and design patterns**

The domain model (Figure 4) uses an abstract User class with four concrete subclasses. Each subclass overrides hasPermission(), so the role check is done by polymorphism rather than by a

switch statement scattered through the code. The service layer (Figure 5) shows the application services, repository interfaces and their SQLite implementations.

Pattern	Where	Why it was chosen
Strategy	LatePenaltyPolicy interface, StandardLatePenaltyPolicy	The penalty rule is the part most likely to change. Services only know the interface, so a new rule is a new class.
Repository	I*Repository interfaces, Sqlite*Repository classes	Keeps all SQL in one layer and lets services be tested with an in-memory database.
Factory	UserFactory::create()	One place turns the role stored in the database into the correct User subclass.
Dependency Injection	Constructors, wired in AppContext	Tests can swap in a fixed clock, a fast hasher and an in-memory database.

SOLID: each service has one job (Single Responsibility); new penalty rules are added without changing services (Open/Closed); every User subclass can be used where a User is expected (Liskov); repositories are split by entity so no service depends on methods it does not use (Interface Segregation); and services depend on interfaces, not on SQLite classes (Dependency Inversion).

Figure 4. Class diagram 1 - domain model, User hierarchy, Strategy and Factory. View diagram ↗️

Figure 5. Class diagram 2 - services, repositories and dependency injection. View diagram ↗️

- **Behavioural models**

The state machine (Figure 6) shows the life cycle of a submission and its mark: SUBMITTED or LATE, then AWAITING_MODERATION, then APPROVED or RETURNED, and finally RELEASED. Guards such as "before deadline and unmarked" are the same checks the code makes. The activity diagram (Figure 7) uses swimlanes for the Student, System, Lecturer and HoD and covers the whole path from submission to released mark, including the return-for-correction loop and the refused release. The two sequence diagrams (Figures 8 and 9) follow the real method calls. The submit sequence shows the first-submission, resubmission and rejected-resubmission paths, and the moderation sequence shows the department check and the all-or-nothing release.

Figure 6. State machine - submission and mark life cycle. View diagram ↗️

Figure 7. Activity diagram with swimlanes - submission to released mark. View diagram ↗️

Figure 8. Sequence diagram - Submit assignment (late and resubmission paths). View diagram ↗️

Figure 9. Sequence diagram - Moderate and release marks. View diagram ↗️

- **Database design**

The database has 13 tables in Third Normal Form (Figure 10). Every non-key column depends only on its table's key: for example a mark's final value is stored in marks, while the moderation history is in its own table instead of repeated columns. Many-to-many links (lecturer to course, student to course) use junction tables with composite keys. CHECK constraints limit statuses and

roles to valid values, and two triggers block UPDATE and DELETE on audit_logs. The data dictionary and full SQL schema are in the Phase 3 document and in src/data/Schema.cpp.

Class	Table	Note
User (+ subclasses)	users	One table with a role column; UserFactory rebuilds the right subclass
Submission / SubmissionVersion	submissions / submission_versions	Versions in their own table so old ones are kept
Mark / ModerationDecision	marks / moderation_decisions	One mark per submission; many decisions per mark
Extension / ClassExtension	extensions / class_extensions	class_extensions added in v1.1 (CR1)
AuditRecord	audit_logs	Append-only through triggers
(association)	lecturer_courses, enrolments	Junction tables, no class needed

Figure 10. Entity relationship diagram (schema version 2). View diagram ↗️

- **Differences from the Phase 3 design**

Three small changes were made during implementation. AuthorizationService and PermissionPolicy were merged into one PermissionPolicy class because they had the same job. A DeadlineResolver class was added because three services needed the same effective-deadline and late-day calculation, and repeating it would have broken the single-responsibility rule. Finally, the class_extensions table was added for CR1 (Section 10).

6. Implementation

The system is about 3,800 lines of C++17 in 55 source files, built with CMake. Everything except main() is compiled into a library (mlms_core), so the tests link exactly the same code that the program runs.

- **Structure**

Folder	Contents
src/ui	Console (safe input reading) and ConsoleUI (one menu per role)
src/services	Auth, UserManagement, CourseManagement, Assignment, Extension, Submission, Marking, Moderation, Release, Audit, PermissionPolicy, DeadlineResolver
src/domain	User hierarchy and UserFactory, LatePenaltyPolicy, entities, DateTime, Validation, exception classes
src/data	Repository interfaces, SQLite repositories, Database/Statement RAII wrappers, TransactionManager, schema migrations
src/security	IPasswordHasher and SodiumPasswordHasher
src/app	AppContext (composition root) and demo data
tests, tools	GoogleTest suite and the 800-student performance test

Repository: Source code and build guide (README) ↗️

- **Key business rules**
  - **Late penalty:** lateDays() rounds up started 24-hour periods. apply() deducts 10% of the maximum mark per day for days 1-3, gives 0 after day 3, and never goes below 0. Example from the demo: 40/50 submitted 2 days late gives a penalty of 10 and a final mark of 30.
  - **Resubmission:** allowed only before the effective deadline and while unmarked. A new version is added and the old ones are kept.
  - **Marking:** only possible after the student's effective deadline, so a student is never locked out of resubmitting.
  - **Moderation and release:** the HoD can only act on courses in their own department. Release is refused as a whole if any submission is unmarked or not approved, so there are no partial releases.
  - **Extensions:** one per student per assignment, plus one class extension (v1.1). The later deadline wins.
- **Security**

Passwords are hashed with libsodium's crypto_pwhash_str, which uses Argon2id with a random salt per password and stores the salt and settings inside the hash. libsodium was chosen because it is a well-reviewed library, Argon2id is memory-hard (slow to attack with GPUs), and hashing and checking are one call each, which leaves little room for mistakes. After 5 failed logins an account is locked for 15 minutes, and the error message is the same for a wrong username and a wrong password. Every service checks both the role and the resource scope (my course, my department, my own records), so hiding a menu option is never the only protection.

- **Persistence and error handling**

SQLite is accessed only through the repository classes, and every query uses bound parameters, so user input is never joined into SQL. Foreign keys are switched on for every connection. Multi-step changes, such as saving a mark together with its audit record, run inside TransactionManager, which commits on success and rolls back if anything throws. Errors are reported with exceptions derived from one base class (MlmsError); the console catches them, shows a clear message and returns to the menu, so bad input or a database error never crashes the program. The code follows a documented coding standard.

Evidence: Coding standard ↗️

7. Verification and Validation

- **Test strategy**

V-Model level	Verifies	How	Exit criterion
Unit	Detailed design: penalty policy, date parsing, User hierarchy	GoogleTest, no database	All pass; business-logic coverage >= 70%
Integration	Architecture: services + repositories + SQLite	In-memory database, fixed clock	All pass; every Must FR has a test
System	Requirements, through UC-01 to UC-03 and their exception flows	Scripted keyboard input into the real console	All pass; NFR-01 under 2 s

V-Model level	Verifies	How	Exit criterion
Acceptance	Stakeholder needs	Walkthrough with the demo data, one session per role	Every checklist item passed

Equivalence partitioning and boundary values were used for the riskiest rules. Submission time was split into on time, late within 3 days and more than 3 days late. Boundary tests cover 1 second before the deadline, exactly on it, 1 second after, exactly 24 hours, 24 hours plus 1 second, 72 hours and 72 hours plus 1 second. Marks were tested at -0.5, 0, the maximum and just above the maximum.

Evidence: Test plan ↗️

- **Automated tests and coverage**

There are 128 automated tests: unit tests for the penalty rules and dates, integration tests for every service, and 11 system tests that drive the real console with scripted input (for example a full run from late submission to released mark, the HoD returning a mark, and a lecturer from another course being denied). All 128 pass, both in CI and on the student's own Windows laptop. Line coverage of the business-logic layer (src/domain and src/services), measured with gcovr, is **95.9%** (902 of 941 lines), well above the 70% target. The performance test builds a database with 800 students and measured an 80 ms login and a 0.5 ms assignment list, against the 2-second target in NFR-01.

Evidence: Full test results v1.1 ↗️

Evidence: Screenshot: 128 tests passing on Windows ↗️

- **System and acceptance testing**

Each system test is linked to a use case flow, such as UC-01 A3 (late submission) or UC-03 E1 (wrong department). An end-to-end demo run is also kept as a console transcript; it shows a submission 2 days late receiving 40 - 10 = 30, being approved and released, and the audit log showing who, when, the old value, the new value and the reason.

Evidence: Console transcript of the full demo ↗️

- **Defect log**

Ten defects were recorded, with severity, cause, fix and retest result. None was critical. Examples: an empty catch block in the rollback code that could hide errors (D-01, Major); integer multiplication that could overflow in date maths (D-02, Major); a test with no assertions (D-07, Major); and a missing cppcheck suppression that made the v1.1 CI run fail and was then fixed (D-10). All defects are closed.

Evidence: Defect log ↗️

8. Configuration Management and Quality Assurance

- **Configuration management**

Git was used with a feature-branch strategy: each piece of work was done on a feature/<name> branch and merged into main with a no-fast-forward merge, so the history shows each feature as one unit. Commit messages follow "area: what changed" and name the requirement or change

request. Releases are tagged v1.0 and v1.1. A GitHub Actions pipeline runs on every push, pull request and tag. It builds the project, runs all tests and the performance check, runs cppcheck and clang-tidy (failing on any finding), measures coverage (failing below 70%) and reports metrics (Figure 11). The pipeline proved its worth when it failed on the first v1.1 push (D-10); the fix was pushed and the run went green.

Figure 11. Configuration management and CI pipeline. View diagram ↗️

Evidence: CI runs on GitHub Actions ↗️ Configuration management plan ↗️

- **Quality assurance**

cppcheck initially reported 23 findings and clang-tidy 13. All were fixed or, in two cases, suppressed with a written reason, leaving zero. The fixes included the D-01 to D-06 defects above. Code metrics were measured with lizard: about 3,800 lines of code in 287 functions, with an average cyclomatic complexity of 2.5. The only function above 15 is the administrator menu switch (16), which was kept because splitting a flat menu would make it harder to read; parseDateTime was reduced from 16 to 4 by splitting it. A self code review against a 15-point checklist (layering, parameterised SQL, permission checks, transactions, validation, const-correctness, test quality) led to the changes recorded in the review.

Evidence: QA report (static analysis and metrics) ↗️

Evidence: Self code review checklist ↗️

9. Deployment

The deployment diagram (Figure 12) shows one school computer running the mlms executable with the SQLite and libsodium libraries, a local mlms.db file and a weekly backup copy on a USB drive. No network connection is needed. The installation guide is written for a school technician and covers installing the tools on Windows (MSYS2) or Ubuntu, building and testing, the first-run administrator setup, the single-computer limit, weekly backup, restore and upgrading. The user manual has one section per role, and the release notes describe v1.0 and v1.1, including known limitations. Release v1.0 is tagged and published on GitHub.

Figure 12. Deployment diagram. View diagram ↗️

Documents: Installation guide ↗️ User manual ↗️ Release notes ↗️

10. Maintenance - Change Request CR1

CR1 asked that "a lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student's assignment list."

- **Classification**

This is **perfective maintenance**. The system was working correctly and its environment had not changed, so it is not corrective or adaptive. The users asked for a new capability that makes the system fit their needs better, which is the definition of perfective maintenance.

- **Impact analysis**

Affected item	Change
Requirements	New FR-21 (class extension); FR-07 now shows extensions; FR-09 penalty starts from the class deadline
Diagrams	Use case, class (ClassExtension), ERD (class_extensions), submit sequence; state and deployment unchanged
Classes	ClassExtension, DeadlineResolver (latest deadline wins), ExtensionService.grantClass, AssignmentService, ConsoleUI
Tables	New class_extensions table; schema version 1 -> 2 with automatic upgrade
Tests	12 new tests; all 116 existing tests rerun as regression

- **Implementation and regression testing**

The change was made on its own branch. A class extension is one row per assignment, so students enrolled later also receive it. The student's effective deadline is the latest of the original, class and individual deadlines, so a class extension can never shorten a student's personal extension. A database created by v1.0 is upgraded automatically when v1.1 opens it, and this was checked with a real v1.0 database file with no data lost. After the change all **128 tests pass: the 116 original tests unchanged plus 12 new ones**. The release was tagged v1.1.

Evidence: CR1 impact analysis and regression results ↗️

11. Professional, Legal and Ethical Issues

- **Data protection**

The system stores personal data about learners, most of whom are minors, including names, student numbers and marks. Lesotho's Data Protection Act 2011 requires that personal data is collected for a clear purpose, kept accurate and protected from unauthorised access. The design supports this through least privilege (the administrator cannot see marks; students see only their own released results), scoped audit access, no plain-text passwords, and an offline database that is not exposed to the internet. The school still has to protect the computer and the backup drive.

- **Security and integrity of assessment records**

Marks affect learners' futures, so they must not be changed without a trace. Every mark entry, correction, moderation decision and release records the actor, time, old value, new value and reason, and the database itself refuses to update or delete audit rows. Transactions make sure a mark is never saved without its audit record.

- **Accessibility**

The console interface works with screen readers and keyboard-only use, and uses plain-language messages. However, it offers no adjustable text size or colour contrast settings. A graphical interface that follows accessibility guidelines would be a sensible future improvement.

- **Fairness of the late-penalty rules**

The penalty is the same for every student and is calculated by one tested class, so it cannot be applied inconsistently by hand. Late work is always accepted and receives feedback, even after day 3, so students still learn from it. Extensions require a written reason that the HoD can see,

which discourages favouritism. One fairness risk remains: the system trusts the school computer's clock, so the technician must keep it correct.

- **Professional responsibility**

Demo accounts and passwords are for evaluation only. Tool licences were checked before use, and the system's limits (single computer, visible password typing) are stated honestly in the release notes rather than hidden.

12. Conclusion

The project delivered a working moderated LMS that meets all Must requirements, follows its design, and is supported by measured evidence: 128 passing tests, 95.9% business-logic coverage, zero static-analysis findings, an 80 ms login with 800 students, and a green CI pipeline. Resolving the stakeholder conflicts carefully in Phase 2 paid off, because rules such as the started-day penalty and marking only after the deadline turned directly into code and tests.

The main limitations are that v1.x runs on one computer only, submissions are stored as text or a file path rather than managed files, passwords are visible while typed, and the release-notification feature (FR-20) was not built. With more time I would add hidden password input, managed file storage, a graphical interface with accessibility options, and a client-server version for a school network. The most useful lesson was that a CI pipeline and a traceability matrix are worth setting up early: the pipeline caught a real defect after release, and the matrix made it easy to see exactly what CR1 affected.

13. References

Boehm, B.W. (1991) 'Software risk management: principles and practices', IEEE Software, 8(1), pp. 32-41.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design patterns: elements of reusable object-oriented software. Reading, MA: Addison-Wesley.

Google (n.d.) GoogleTest user guide. Available at: https://google.github.io/googletest/ (Accessed: 1 November 2026).

ISO/IEC/IEEE (2018) ISO/IEC/IEEE 29148:2018 Systems and software engineering - Life cycle processes - Requirements engineering. Geneva: ISO.

Kingdom of Lesotho (2012) Data Protection Act, 2011 (Act No. 5 of 2012). Maseru: Government Gazette.

Kitware (n.d.) CMake documentation. Available at: https://cmake.org/documentation/ (Accessed: 1 November 2026).

libsodium (n.d.) Password hashing (crypto_pwhash). Available at: https://doc.libsodium.org/ (Accessed: 1 November 2026).

Martin, R.C. (2017) Clean architecture: a craftsman's guide to software structure and design. Boston: Prentice Hall.

Project Management Institute (2021) A guide to the project management body of knowledge (PMBOK guide). 7th edn. Newtown Square, PA: PMI.

Sommerville, I. (2016) Software engineering. 10th edn. Harlow: Pearson.

SQLite (n.d.) Appropriate uses for SQLite. Available at: https://www.sqlite.org/whentouse.html (Accessed: 1 November 2026).

14. Appendices

The appendices are kept in the repository so that they stay up to date with the code. Each link opens the document on GitHub.

Appendix	Contents	File in repository
A	Requirements traceability matrix (FR/NFR -> use case -> classes -> tables -> source files -> tests)	docs/ TRACEABILITY.md
B	Full test results v1.0 and v1.1	docs/ test_results_v1.1.txt
C	User manuals for all four roles	docs/ USER_MANUAL.md
D	Updated risk register (Gates 0-3)	docs/ RISK_REGISTER.md
E	All diagrams with editable PlantUML sources	docs/diagrams/
F	Phase 1-3 documents	docs/phases/

- Appendix A - Traceability matrix ↗️
- Appendix B - Test results v1.1 ↗️
- Appendix B - Test results v1.0 ↗️
- Appendix C - User manual ↗️
- Appendix D - Risk register ↗️
- Appendix E - Diagrams ↗️
- Appendix F - Phase documents ↗️